

Pixel Battle — Ranged Combat & Mobility (Sub-project C) Implementation Plan

Pixel Battle — Ranged Combat & Mobility (Sub-project C) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Stop the hitstop stutter, give projectile skills real long range, add a Flash mobility ability, shrink the on-screen characters, and re-author the script library so a mage genuinely kites.

Architecture: Five engine/renderer changes plus a script re-author. Hitstop becomes conditional (significant hits only). `Skill.range` gains numeric support via an `effective_range` accessor. Flash is two additive engine action ints (9/10) the `ScriptDriver` can emit. All renderer-side / engine-data changes — no RL retrain.

Tech Stack: Python 3.10, pygame (headless SDL dummy), PyYAML, pytest, ffmpeg.

Design spec: docs/superpowers/specs/2026-05-22-pixel-battle-ranged-mobility-design.md

File Structure

File	Status	Responsibility
pixel_battle/engine/battle.py	modify	<code>_hit_causes_hitstop</code> helper; hitstop only on significant hits; <code>_resolve_attack_hit</code> uses <code>skill.effective_range</code> ; <code>EventType.FLASH</code>
pixel_battle/engine/skill.py	modify	<code>Skill.range</code> accepts a number; <code>effective_range</code> accessor
pixel_battle/engine/physics.py	modify	<code>MAX_ATTACK_RANGE</code> , <code>FLASH_DISTANCE</code> , <code>FLASH_COOLDOWN_MS</code> constants

File	Status	Responsibility
<code>pixel_battle/engine/character.py</code>	modify	<code>flash_ready_at_ms</code> cooldown field
<code>pixel_battle/data/characters.json</code>	modify	numeric range on projectile skills
<code>pixel_battle/rl/env.py</code>	modify	<code>_apply_action</code> widens the cd/special gate; Flash actions 9/10 + <code>_do_flash</code>
<code>pixel_battle/script/loader.py</code>	modify	<code>flash:in</code> / <code>flash:back</code> verbs in <code>D0_VERBS</code>
<code>pixel_battle/rl/stick_renderer.py</code>	modify	<code>spawn_flash_puff</code> VFX; <code>get_style</code> applies <code>_FIGURE_SCALE</code>
<code>pixel_battle/rl/play.py</code>	modify	<code>CAM_ZOOM</code> → 1.0; spawn the Flash puff on a FLASH event
<code>pixel_battle/data/scripts/*.yaml</code>	modify	re-author all 5 to kite at range + use Flash
<code>pixel_battle/tests/test_*.py</code>	new/modify	per-task tests

No RL retrain: Flash is engine action ints 9/10 — additive; the RL `Discrete(9)` model never emits them. `characters.json` gains data-only range fields. Observation/action space/reward untouched.

Action integer map (after this sub-project): 0 idle, 1 retreat, 2 advance, 3 jump, 4 basic, 5 cd, 6 ultimate, 7 special, 8 kick, 9 flash:in, 10 flash:back.

Task 1: Hitstop fires only on significant hits

Files: - Modify: `pixel_battle/engine/battle.py` - Modify: `pixel_battle/tests/test_hitstop.py`

`battle.py` current facts: `_hitstop_for(is_crit)` helper exists. `_resolve_attack_hit` computes `is_crit` and, near the end, does `self._hitstop_remaining = _hitstop_for(is_crit)` unconditionally. `SkillType` is imported (enum: BASIC/COOLDOWN/SPECIAL/ULTIMATE). The current `test_hitstop.py::test_a_landed_hit_sets_hitstop` calls `_resolve_attack_hit` with a basic skill and asserts a hitstop was set — that assertion becomes wrong under this task and is replaced below.

☐ Step 1: Write the failing test

Replace `test_a_landed_hit_sets_hitstop` in `pixel_battle/tests/test_hitstop.py` with these tests (keep the other tests in that file, and the existing imports — add `_hit_causes_hitstop` to the `from pixel_battle.engine.battle import ...` line and `from pixel_battle.engine.skill import SkillType`):

```

def test_hitstop_skips_plain_basic_hits():
    # A non-crit basic hit must NOT trigger hitstop – basic spam should not stutter.
    assert _hit_causes_hitstop(False, SkillType.BASIC) is False

def test_hitstop_fires_on_crit_basic():
    assert _hit_causes_hitstop(True, SkillType.BASIC) is True

def test_hitstop_fires_on_skill_hits():
    assert _hit_causes_hitstop(False, SkillType.COOLDOWN) is True
    assert _hit_causes_hitstop(False, SkillType.SPECIAL) is True
    assert _hit_causes_hitstop(False, SkillType.ULTIMATE) is True

def test_special_hit_sets_hitstop_via_resolve():
    from pixel_battle.rl.env import PixelBattleEnv
    from pixel_battle.engine.skill import SkillType as _ST
    env = PixelBattleEnv(seed=1)
    b = env.battle
    atk, dfn = env.left, env.right
    atk.pos_x, dfn.pos_x = 240.0, 300.0
    atk.accuracy = 1.0
    special = atk.skills_of_type(_ST.SPECIAL)[0]
    atk.attack_used_kind = special
    atk.mp = 100
    b._hitstop_remaining = 0
    b._resolve_attack_hit(atk, dfn)
    assert b._hitstop_remaining > 0          # a special hit always freezes

```

☐ Step 2: Run the tests to verify they fail

Run: `python -m pytest pixel_battle/tests/test_hitstop.py -v` Expected: FAIL — `ImportError: cannot import name '_hit_causes_hitstop'.`

☐ Step 3: Add the `_hit_causes_hitstop` helper

In `pixel_battle/engine/battle.py`, next to `_hitstop_for`, add:

```

def _hit_causes_hitstop(is_crit: bool, skill_type: SkillType) -> bool:
    """Hitstop fires only on a crit or a non-basic skill hit. Plain basic

```

```
hits do not freeze the sim, so rapid basic trading renders smoothly."""
return is_crit or skill_type is not SkillType.BASIC
```

☐ Step 4: Make `_resolve_attack_hit` conditional

In `_resolve_attack_hit`, find the line `self._hitstop_remaining = _hitstop_for(is_crit)` and replace it with:

```
if _hit_causes_hitstop(is_crit, skill.skill_type):
    self._hitstop_remaining = _hitstop_for(is_crit)
```

(`skill` and `is_crit` are already in scope at that point.)

☐ Step 5: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_hitstop.py -v` Expected: PASS.

☐ Step 6: Run the engine regression tests

Run: `python -m pytest pixel_battle/tests/test_battle_no_lock.py pixel_battle/tests/test_effect_application.py -v` Expected: all green.

☐ Step 7: Commit

```
git add pixel_battle/engine/battle.py pixel_battle/tests/test_hitstop.py
git commit -m "feat(pixel-battle/engine): hitstop only on significant hits - no basic-spam stutter"
```

Task 2: Per-skill numeric range

Files: - Modify: `pixel_battle/engine/physics.py` - Modify: `pixel_battle/engine/skill.py` - Modify: `pixel_battle/engine/battle.py` - Modify: `pixel_battle/rl/env.py` - Modify: `pixel_battle/data/characters.json` - Test: `pixel_battle/tests/test_skill_range.py` (new)

`physics.py` has `MELEE_RANGE = 110`, `SPECIAL_RANGE = 130`. `skill.py`: `Skill` is a dataclass with `range: str = "melee"`, `from_dict` reads `range=d.get("range", "melee")`. `battle.py`'s `_resolve_attack_hit` has a 3-branch string check that sets `range_limit`. `env.py`'s `_apply_action` gates actions 5/7 with `dist <= SPECIAL_RANGE`.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_skill_range.py
"""Per-skill numeric attack range."""
```

```

from pixel_battle.engine.skill import Skill, SkillType
from pixel_battle.engine.physics import MELEE_RANGE, SPECIAL_RANGE

def test_effective_range_numeric():
    s = Skill.from_dict({"id": "x", "type": "cooldown", "anim": "a",
                        "range": 280})
    assert s.effective_range == 280

def test_effective_range_string_special():
    s = Skill.from_dict({"id": "x", "type": "cooldown", "anim": "a",
                        "range": "special"})
    assert s.effective_range == SPECIAL_RANGE

def test_effective_range_special_type_defaults_to_special():
    # A SPECIAL-type skill with no explicit range still reaches SPECIAL_RANGE.
    s = Skill.from_dict({"id": "x", "type": "special", "anim": "a"})
    assert s.effective_range == SPECIAL_RANGE

def test_effective_range_basic_defaults_to_melee():
    s = Skill.from_dict({"id": "x", "type": "basic", "anim": "a"})
    assert s.effective_range == MELEE_RANGE

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_skill_range.py -v` Expected: FAIL — Skill has no `effective_range`.

☐ Step 3: Add `MAX_ATTACK_RANGE` to `physics.py`

In `pixel_battle/engine/physics.py`, in the Combat section (next to `SPECIAL_RANGE`):

```
MAX_ATTACK_RANGE = 360      # upper bound for the env pre-fire gate on cd/special
```

☐ Step 4: Add `effective_range` to `Skill`

In `pixel_battle/engine/skill.py`: add the import and the accessor. Add to the top imports:

```
from typing import Optional, Union
```

```
from pixel_battle.engine.physics import MELEE_RANGE, SPECIAL_RANGE
```

Change the `range` field annotation in the `Skill` dataclass from `range: str = "melee"` to:

```
range: Union[str, int] = "melee"
```

Add this property to the `Skill` class:

```
@property
def effective_range(self) -> int:
    """Numeric attack range in px — a numeric `range` wins; otherwise
    'special'/SPECIAL-type skills reach SPECIAL_RANGE, the rest MELEE_RANGE."""
    r = self.range
    if isinstance(r, (int, float)) and not isinstance(r, bool):
        return int(r)
    if r == "special" or self.skill_type is SkillType.SPECIAL:
        return SPECIAL_RANGE
    return MELEE_RANGE
```

☐ Step 5: Use `effective_range` in `_resolve_attack_hit`

In `pixel_battle/engine/battle.py`'s `_resolve_attack_hit`, replace the range-limit block — the 3-branch `if skill.range == "special": ... elif skill.skill_type is SkillType.SPECIAL: ... else: ...` that sets `range_limit` — with a single line:

```
range_limit = skill.effective_range
```

☐ Step 6: Widen the env pre-fire gate

In `pixel_battle/rl/env.py`: add `MAX_ATTACK_RANGE` to the `from pixel_battle.engine.physics import ...` line. In `_apply_action`, change the cd and special gates from `dist <= SPECIAL_RANGE` to `dist <= MAX_ATTACK_RANGE`:

```
elif action == 5 and dist <= MAX_ATTACK_RANGE:    # cd skill
    self.battle._start_attack_with_kind(me, opp, "cooldown")
    ...
elif action == 7 and dist <= MAX_ATTACK_RANGE:    # special skill
    self.battle._start_attack_with_kind(me, opp, "special")
```

(Actions 4/8 — basic/kick — stay at `MELEE_RANGE`. The authoritative per-skill range check remains `_resolve_attack_hit`.)

☐ Step 7: Add numeric range data to characters.json

In `pixel_battle/data/characters.json`, add a numeric "range" field to these projectile skills (find each skill object by `id` and add the key):

Character	Skill id	"range"
lux	light_binding	280
lux	lucent_singularity	300
lux	final_spark	340
ashe	volley	260
ashe	hawkshot	300
ashe	enchanted_crystal_arrow	340
brick_phone	screw_dart	250
brick_phone	snake_strike	240
glass_slab	shard_scatter	240
glass_slab	ad_popup_spam	240
glass_slab	force_update	320

Leave every other skill unchanged (Garen and Yasuo skills, all basics, and self-buff skills keep their short range).

☐ Step 8: Run the tests + regression

Run: `python -m pytest pixel_battle/tests/test_skill_range.py`

`pixel_battle/tests/test_lol_champions.py pixel_battle/tests/test_battle_no_lock.py -v`

Expected: all green (`characters.json` still loads; range logic works).

☐ Step 9: Commit

```
git add pixel_battle/engine/physics.py pixel_battle/engine/skill.py
      pixel_battle/engine/battle.py pixel_battle/rl/env.py
      pixel_battle/data/characters.json pixel_battle/tests/test_skill_range.py
git commit -m "feat(pixel-battle/engine): per-skill numeric range - true long-range skills"
```

Task 3: Flash mobility ability

Files: - Modify: `pixel_battle/engine/physics.py` - Modify: `pixel_battle/engine/character.py` -
Modify: `pixel_battle/engine/battle.py` - Modify: `pixel_battle/rl/env.py` - Modify:

pixel_battle/script/loader.py - Test: pixel_battle/tests/test_flash.py (new)

character.py: Character is a dataclass; reset_physics resets runtime fields; last_attack_ms is an existing cooldown-style int field. battle.py: EventType is an Enum. env.py: _apply_action maps action ints; clamp_x is importable from physics.py.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_flash.py
"""Flash mobility ability."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.rl.env import PixelBattleEnv
from pixel_battle.engine.physics import FLASH_DISTANCE, FLASH_COOLDOWN_MS, clamp_x
from pixel_battle.script.loader import DO_VERBS

def test_flash_verbs_registered():
    assert DO_VERBS["flash:in"] == 9
    assert DO_VERBS["flash:back"] == 10

def test_flash_back_moves_away_from_opponent():
    env = PixelBattleEnv(seed=1)
    me, opp = env.left, env.right
    me.pos_x, opp.pos_x = 240.0, 360.0      # opponent is to the right
    me.flash_ready_at_ms = 0
    env._apply_action(me, opp, 10)         # flash:back
    assert me.pos_x == clamp_x(240.0 - FLASH_DISTANCE)  # blinked left, away

def test_flash_in_moves_toward_opponent():
    env = PixelBattleEnv(seed=1)
    me, opp = env.left, env.right
    me.pos_x, opp.pos_x = 240.0, 360.0
    me.flash_ready_at_ms = 0
    env._apply_action(me, opp, 9)          # flash:in
    assert me.pos_x == clamp_x(240.0 + FLASH_DISTANCE)  # blinked right, toward

def test_flash_respects_cooldown():
```



```

env = PixelBattleEnv(seed=1)
me, opp = env.left, env.right
me.pos_x, opp.pos_x = 240.0, 360.0
me.flash_ready_at_ms = 0
env._apply_action(me, opp, 10)           # first flash – fires
after_first = me.pos_x
env._apply_action(me, opp, 10)           # immediately again – on cooldown, no-op
assert me.pos_x == after_first
assert me.flash_ready_at_ms > env.battle.elapsed_ms

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_flash.py -v` Expected: FAIL — `FLASH_DISTANCE` does not exist / `flash:in` not in `DO_VERBS`.

☐ Step 3: Add Flash constants to `physics.py`

In `pixel_battle/engine/physics.py`, in the Motion section:

```

FLASH_DISTANCE = 130           # px a Flash teleports
FLASH_COOLDOWN_MS = 3500       # Flash recharge time

```

☐ Step 4: Add the Flash cooldown field to `Character`

In `pixel_battle/engine/character.py`: add a field to the `Character` dataclass (next to `last_attack_ms`):

```
flash_ready_at_ms: int = 0
```

In `reset_physics`, add (next to the other resets):

```
self.flash_ready_at_ms = 0
```

☐ Step 5: Add the `FLASH` event type

In `pixel_battle/engine/battle.py`, add to the `EventType` enum:

```
FLASH = "flash"
```

☐ Step 6: Add the Flash actions to `_apply_action`

In `pixel_battle/rl/env.py`'s `_apply_action`, after the `action == 8` (kick) branch, add:

```

elif action == 9:                                # flash toward opponent
    self._do_flash(me, opp, toward=True)
elif action == 10:                               # flash away from opponent
    self._do_flash(me, opp, toward=False)

```

Add the `_do_flash` helper method to the `env` class (next to `_apply_action`). Ensure `clamp_x` and `FLASH_DISTANCE / FLASH_COOLDOWN_MS` are imported from `physics.py`, and `EventType` from `battle.py`:

```

def _do_flash(self, me: Character, opp: Character, toward: bool) -> None:
    """Instant teleport (Flash). No-op while on cooldown."""
    if self.battle.elapsed_ms < me.flash_ready_at_ms:
        return
    sign = 1 if opp.pos_x > me.pos_x else -1
    if not toward:
        sign = -sign
    from_x = me.pos_x
    me.pos_x = clamp_x(me.pos_x + sign * FLASH_DISTANCE)
    me.facing = 1 if opp.pos_x > me.pos_x else -1
    me.flash_ready_at_ms = self.battle.elapsed_ms + FLASH_COOLDOWN_MS
    self.battle._emit(EventType.FLASH, actor=me.id,
                      extra={"from_x": from_x, "to_x": me.pos_x})

```

☐ Step 7: Register the Flash verbs

In `pixel_battle/script/loader.py`, add to the `D0_VERBS` dict:

```

"flash:in": 9, "flash:back": 10,

```

☐ Step 8: Run the tests + regression

Run: `python -m pytest pixel_battle/tests/test_flash.py pixel_battle/tests/test_script_loader.py pixel_battle/tests/test_env_attack_gate.py -v`
 Expected: all green.

☐ Step 9: Commit

```

git add pixel_battle/engine/physics.py pixel_battle/engine/character.py
      pixel_battle/engine/battle.py pixel_battle/rl/env.py
      pixel_battle/script/loader.py pixel_battle/tests/test_flash.py
git commit -m "feat(pixel-battle): Flash mobility ability – scriptable instant teleport"

```

Task 4: Flash VFX

Files: - Modify: `pixel_battle/rl/stick_renderer.py` - Modify: `pixel_battle/rl/play.py` - Test: `pixel_battle/tests/test_flash_vfx.py` (new)

`play.py`'s `_render_fight` has a per-frame event loop (`for ev in env.battle.events[prev_ev_n:]` , `et = ev.type.value`) that already branches on event types and spawns VFX.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_flash_vfx.py
"""Flash blink VFX."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

import numpy as np
import pygame
import pytest

from pixel_battle.rl.stick_renderer import spawn_flash_puff

@pytest.fixture(scope="module", autouse=True)
def _pg():
    pygame.init()
    pygame.display.set_mode((1, 1))
    yield
    pygame.quit()

def test_flash_puff_marks_the_surface():
    surf = pygame.Surface((480, 854))
    surf.fill((0, 0, 0))
    spawn_flash_puff(surf, 240, 530, (180, 210, 255))
    arr = pygame.surfarray.array3d(surf)
    assert (arr > 0).any()
```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_flash_vfx.py -v` Expected: FAIL — `spawn_flash_puff` does not exist.

☐ Step 3: Add `spawn_flash_puff` to `stick_renderer.py`

Add this function to `pixel_battle/rl/stick_renderer.py` (next to `spawn_impact_burst` / `spawn_landing_dust`):

```
def spawn_flash_puff(surf, x: int, ground_y: int, color) -> None:
    """A quick blink mark – a bright vertical streak plus a few outward
    sparks – drawn at a Flash origin or destination."""
    x, ground_y = int(x), int(ground_y)
    top = ground_y - 150
    # Vertical light streak.
    streak = pygame.Surface((10, 150), pygame.SRCALPHA)
    pygame.draw.line(streak, (color[0], color[1], color[2], 150),
                     (5, 0), (5, 150), 4)
    surf.blit(streak, (x - 5, top))
    # Outward spark dots.
    for dx, dy in ((-14, -40), (14, -40), (-10, -90), (10, -90)):
        pygame.draw.circle(surf, color, (x + dx, ground_y + dy), 3)
```

☐ Step 4: Spawn the puff on a FLASH event

In `pixel_battle/rl/play.py`: add `spawn_flash_puff` to the existing `from pixel_battle.rl.stick_renderer import (...)` import. In `_render_fight`'s per-frame event loop, add a branch for the flash event (alongside the other `et == ...` branches):

```
elif et == "flash":
    fx = ev.extra or {}
    ground_y = env.left.pos_y      # both fighters share the floor
    actor_col = lcol if ev.actor == env.left.id else rcol
    spawn_flash_puff(world, fx.get("from_x", 0), ground_y, actor_col)
    spawn_flash_puff(world, fx.get("to_x", 0), ground_y, actor_col)
```

(Match the indentation and the `et` / `world` / `lcol` / `rcol` names already used in that loop — read the surrounding branches to confirm.)

☐ Step 5: Run the test + regression

Run: `python -m pytest pixel_battle/tests/test_flash_vfx.py`
`pixel_battle/tests/test_play_richness.py pixel_battle/tests/test_skill_vfx.py -v` Expected: all green.

☐ Step 6: Commit

```
git add pixel_battle/rl/stick_renderer.py pixel_battle/rl/play.py
      pixel_battle/tests/test_flash_vfx.py
git commit -m "feat(pixel-battle/rl): Flash blink VFX"
```

Task 5: Smaller characters

Files: - Modify: `pixel_battle/rl/play.py` - Modify: `pixel_battle/rl/stick_renderer.py` - Test: `pixel_battle/tests/test_stick_renderer_pose.py`

`stick_renderer.py` has `_STYLES` (per-character size dict) and `get_style(char_id)` which returns the raw `_STYLES` entry (or `_DEFAULT_STYLE`).

☐ Step 1: Write the failing test

Add to `pixel_battle/tests/test_stick_renderer_pose.py`:

```
def test_get_style_applies_figure_scale():
    from pixel_battle.rl.stick_renderer import get_style, _STYLES, _FIGURE_SCALE
    raw = _STYLES["garen"]
    scaled = get_style("garen")
    # head_shape (a string) is untouched; size fields are scaled down.
    assert scaled["head_shape"] == raw["head_shape"]
    assert scaled["torso_length"] == max(1, int(raw["torso_length"] * _FIGURE_SCALE))
    assert _FIGURE_SCALE < 1.0
```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest`

`pixel_battle/tests/test_stick_renderer_pose.py::test_get_style_applies_figure_scale -v`

Expected: FAIL — `_FIGURE_SCALE` does not exist; `get_style` returns unscaled values.

☐ Step 3: Lower `CAM_ZOOM`

In `pixel_battle/rl/play.py`, set `CAM_ZOOM` to `1.0`:

```
CAM_ZOOM = 1.0           # whole arena in frame – best for reading long-range kiting
```

If `pixel_battle/tests/test_poses.py` has a comment quoting `CAM_VIEW_H` derived from a prior `CAM_ZOOM`, update it (`854 / 1.0 = 854`).

☐ Step 4: Apply `_FIGURE_SCALE` in `get_style`

In `pixel_battle/rl/stick_renderer.py`, add the scale constant and rewrite `get_style`:

```
_FIGURE_SCALE = 0.85          # shrink every figure ~15% - characters read smaller
_SCALED_KEYS = ("head_size", "torso_length", "upper_arm", "forearm",
               "thigh", "shin", "line_width", "hand_radius", "foot_length")

def get_style(char_id: str) -> dict:
    base = _STYLES.get(char_id, _DEFAULT_STYLE)
    return {k: (max(1, int(v * _FIGURE_SCALE)) if k in _SCALED_KEYS else v)
           for k, v in base.items()}
```

(If `get_style` currently has a different body, replace it entirely with the above. `head_shape` is not in `_SCALED_KEYS`, so the string passes through untouched.)

☐ Step 5: Run the tests + full pose/render suite

Run: `python -m pytest pixel_battle/tests/test_stick_renderer_pose.py pixel_battle/tests/test_poses.py pixel_battle/tests/test_play_richness.py -v` Expected: all green — the visual-safety lock still passes (smaller figures + a wider camera only add headroom).

☐ Step 6: Commit

```
git add pixel_battle/rl/play.py pixel_battle/rl/stick_renderer.py
      pixel_battle/tests/test_stick_renderer_pose.py
git commit -m "feat(pixel-battle/rl): smaller figures - CAM_ZOOM 1.0 + 15% figure scale"
```

Task 6: Re-author the script library for ranged kiting

Files: - Modify: all 5 files in `pixel_battle/data/scripts/`

With per-skill range and Flash now available, re-author all 5 scripts so the mage scripts genuinely kite.

☐ Step 1: Re-author the scripts

Re-author each of the 5 `pixel_battle/data/scripts/*.yaml` with these principles: - **Mage / ranged side** (Lux, Glass Slab, Ashe): cast cd/special skills from a real distance — author `until: "dist<=270"` (not `<=130`) before an `attack:cd / attack:special`, matching the new skill ranges from Task 2 (e.g. Lux `light_binding` reaches 280, `lucent_singularity` 300). When the bruiser closes inside ~140 px, use `{do: "flash:back", until: "dist>=240"}` to escape and re-establish range. - **Bruiser side** (Garen, Yasuo): close the gap with `advance` and `{do: "flash:in", until: "dist<=120"}`; their skills stay short-range so they must get in. - Keep each script's themed identity and intended winner from its

filename. - Keep the MP economy (specials only after MP is built — `attack:basic / attack:cd` early; characters start at `mp = 0`). - Keep a relentless KO finisher on the intended winner (`{do: "attack:basic", until: "target_hp<=0"}` as the last intent), so every fight still ends in a decisive KO. - Only use `do` verbs in `D0_VERBS` (now including `flash:in / flash:back`) and `until` conditions the compiler supports.

Example of the new kiting pattern (a fragment of a mage's `right_script`):

```
- {do: retreat,          until: "dist>=270"}
- {do: "attack:cd",      until: skill_done}      # roots at long range
- {do: "attack:special", until: skill_done}      # ranged poke
- {do: "flash:back",     until: "dist>=250"}     # escape if crowded
- {do: retreat,          until: "dist>=270"}
- {do: "attack:special", until: skill_done}
```

☐ Step 2: Verify all 5 scripts load

Run: `python -m pytest pixel_battle/tests/test_script_library.py -v` Expected: PASS — all 5 re-authored scripts load + validate. Fix any the loader rejects.

☐ Step 3: Commit

```
git add pixel_battle/data/scripts/
git commit -m "feat(pixel-battle): re-author scripts for true ranged kiting + Flash"
```

Task 7: Full test sweep + validation render

Files: none (verification only; possible tuning commits)

☐ Step 1: Run the entire test suite

Run: `python -m pytest pixel_battle/tests/ -q` Expected: all green. Fix any regression before continuing.

☐ Step 2: Render all 5 scripts

For each `.yaml` in `pixel_battle/data/scripts/`, run: `python -m pixel_battle.rl.play_scripted pixel_battle/data/scripts/<name>.yaml` Expected: each writes an mp4 to `pixel_battle/output/scripted/` with no Python error, ending in a decisive KO (duration well under 60 s).

☐ Step 3: Inspect the output

For each mp4, `ffprobe -v error -show_entries format=duration,size -of default=noprint_wrappers=1 <file>`. Watch them and check against the spec's success criteria: no stutter; characters smaller, arena reads wide; the mage visibly kites at range and Flashes to escape; the bruiser closes the gap; every fight KOs.

☐ Step 4: Tune

Adjust from what the renders show, then re-render: - Still stuttering → check no significant-hit path overfires hitstop. - Characters still too big → lower `CAM_ZOOM` / `_FIGURE_SCALE`. - Mage not kiting well / fights not reading → adjust the script intent distances or the per-skill `range` values. - Flash too short/long or too frequent → adjust `FLASH_DISTANCE` / `FLASH_COOLDOWN_MS`. After any change, re-run `python -m pytest pixel_battle/tests/ -q`.

☐ Step 5: Commit any tuning changes

```
git add -A
git commit -m "tune(pixel-battle): ranged-combat & mobility validation tuning"
```

Self-Review

Spec coverage — every spec section maps to a task: - §4.1 hitstop stutter fix → Task 1 - §4.2 per-skill numeric range → Task 2 - §4.3 Flash ability → Task 3; Flash VFX → Task 4 - §4.4 smaller characters → Task 5 - §4.5 re-author scripts → Task 6 - §5 error handling → Task 2 (`effective_range` fallback), Task 3 (`_do_flash` cooldown no-op + `clamp_x`) - §6 testing → Tasks 1-5 are TDD; Task 7 the full sweep - §7 validation → Task 7 - §3 no retrain → no task touches the RL observation / `Discrete(9)` action space / reward; Flash uses additive action ints 9/10 the RL model never emits

No placeholders — all constants are concrete (`MAX_ATTACK_RANGE=360`, `FLASH_DISTANCE=130`, `FLASH_COOLDOWN_MS=3500`, `_FIGURE_SCALE=0.85`, `CAM_ZOOM=1.0`, the `range` table); Task 6's script re-author gives the concrete kiting pattern + principles; Task 7 is the tuning pass.

Type consistency — checked: `_hit_causes_hitstop(is_crit, skill_type)` (Task 1); `Skill.effective_range` / `Skill.range`: `Union[str,int]` (Task 2) used by `_resolve_attack_hit`; `MAX_ATTACK_RANGE` (physics) used in `env.py`; `FLASH_DISTANCE` / `FLASH_COOLDOWN_MS` (physics), `Character.flash_ready_at_ms`, `EventType.FLASH`, `_do_flash`, `DO_VERBS` `flash:in` / `flash:back` → action ints 9/10 (Task 3) consistent with `spawn_flash_puff` + the `et == "flash"` handler (Task 4); `_FIGURE_SCALE` / `_SCALED_KEYS` / `get_style` (Task 5).